

Candidate Dataset Generation说明

本项目的核心实验文件是：

```
data/processed/candidates.csv
```

它不是普通的医疗问答表，而是一个用于 reranker comparison 的 question-context pair 数据集。每一行表示：

```
一个 medical question + 一个 candidate medical context
```

后续所有方法，包括 `embedding_baseline`、`bge`、`bce` 和 `lightweight_ml`，都必须读取同一个 `candidates.csv`，这样不同方法的排序结果才具有可比性。

1. 数据来源

原始数据来自中文医疗问答 CSV，目前使用两个科室数据：

```
data/raw/oncology_sample.csv
data/raw/pediatric_sample.csv
```

每条原始记录大致包含：

```
department, title, ask, answer
```

其中：

- `ask`：患者问题
- `answer`：医生回答
- `title`：问答标题
- `department`：原始科室或疾病类别

在生成 candidate context 时，我们不是只使用 `answer`，而是将 `title + ask + answer` 组合成一个更完整的候选医疗资料：

```
标题：...
患者问题：...
医生回答：...
```

这样做的原因是，真实 RAG 系统中的 context 通常不只是单句答案，而是包含一定上下文的信息块。

2. 生成目标

我们希望模拟 RAG 中的 Top-K retrieval 场景：

给定一个 question
↓
系统先获得若干条 candidate contexts
↓
reranker 对这些 contexts 重新打分排序
↓
评估真正相关的 context 能否排到更靠前的位置

因此，本项目不直接评估最终医疗回答质量，而是评估：

不同 reranker 是否能把正确 context 排到前面

3. 每个问题的候选构造规则

当前默认设置为：

100 个问题
每个问题 10 条 candidate contexts
总计约 1000 行 question-context pairs

每个问题包含：

1 条 positive
4 条 hard_negative
5 条 random_negative

也就是：

$1 + 4 + 5 = 10$ candidates per question

4. Positive 样本规则

`positive` 表示原始数据中和该 question 配对的 answer。

如果原始 CSV 中某一行是：

```
ask_i → answer_i
```

那么构造 candidate 时:

```
question = ask_i  
candidate_context = title_i + ask_i + answer_i  
label = 1  
source_type = positive
```

这表示该 context 是该问题在原始数据中的真实配对回答。

注意: 这里的 `label=1` 表示“原始问答配对相关”, 不是医学上唯一正确答案。

5. Hard Negative 样本规则

`hard_negative` 表示同科室但不同问题的候选资料。

例如当前 question 来自儿科, 那么 hard negative 会从儿科数据中抽取其他问答记录:

```
question = 儿科问题 A  
candidate_context = 儿科问题 B 的 title + ask + answer  
label = 0  
source_type = hard_negative
```

hard negative 的作用是增加任务难度。

如果只有不同科室的 random negative, 模型可能只学会:

```
科室一样 → 相关  
科室不同 → 不相关
```

这会导致模型过于依赖 department 信息。加入 hard negative 后, 模型必须进一步区分同科室内部不同疾病或不同症状之间的语义差异。

6. Random Negative 样本规则

`random_negative` 表示从另一个科室随机抽取的候选资料。

例如当前 question 来自儿科, 那么 random negative 会从肿瘤科抽取; 如果当前 question 来自肿瘤科, 则从儿科抽取。

```
question = 儿科问题  
candidate_context = 肿瘤科问答资料
```

```
label = 0
source_type = random_negative
```

random negative 的作用是提供较容易区分的负样本，帮助模型学习基本的跨科室不相关关系。

7. 关于 Label 的说明

本项目的 label 是自动构造的 weak supervision，而不是人工医生标注。

规则为：

```
原始 question 对应的原始 answer → label = 1
其他抽样得到的 candidate contexts → label = 0
```

因此，可能存在一种情况：某些 negative context 虽然不是原始配对答案，但仍然包含部分通用医学建议，例如“监测体温”“注意休息”“及时就医”等。

这属于自动构造数据集的正常噪声。我们在实验中将这些样本仍视为 `label=0`，因为本项目评估的是 reranker 能否找回“原始配对 context”，而不是判断所有医学建议是否都有一定合理性。

在 report 中可以将这一点写作 limitation：

```
Because labels are automatically constructed from original QA pairs, some negative samples may still contain partially relevant medical advice. Therefore, the labels should be interpreted as weak supervision rather than human-verified relevance judgments.
```

8. candidates.csv 字段说明

最终生成的 `candidates.csv` 包含以下字段：

```
qid
question
question_department
context_id
candidate_context
label
department
source_type
embedding_score
```

字段含义如下：

字段	含义
qid	问题 ID，例如 q0001
question	当前医疗问题
question_department	当前 question 所属科室，例如 pediatrics 或 oncology
context_id	candidate context ID，例如 c000001
candidate_context	候选医疗资料，由 title、ask、answer 组合而成
label	是否为原始配对答案；1 表示相关，0 表示不相关
department	candidate context 所属科室
source_type	样本来源类型：positive、hard_negative、random_negative
embedding_score	原始 embedding cosine similarity 分数

9. embedding_score 的生成逻辑

embedding_score 表示原始 RAG 检索阶段的相似度分数。

本项目优先复用原 RAG 项目的 embedding 思路：

```
question → nomic-embed-text embedding
candidate_context → nomic-embed-text embedding
embedding_score = cosine(question_embedding, context_embedding)
```

也就是说，embedding_score 本质上是：

```
cosine similarity between question and candidate_context
```

这与原 RAG 系统的 retrieval baseline 保持一致。原 RAG 项目中本身使用 Ollama 的 nomic-embed-text 作为 embedding 模型，并通过向量相似度进行检索。

为了避免重复计算，当前实现中加入了 in-memory embedding cache：相同文本只会调用一次 Ollama embedding，后续重复出现时直接从 cache 中读取。

如果 Ollama、nomic-embed-text 或本地服务不可用，代码会 fallback 到 character-level TF-IDF cosine similarity，保证数据集生成流程不会完全中断。但正式实验中，优先使用 Ollama embedding 结果作为 baseline。

10. 为什么不使用完整原 RAG retrieve 逻辑？

本项目只复用原 RAG 的 embedding + cosine similarity，不复用完整的 retrieve heuristic。

保留：

```
nomiic-embed-text embedding
cosine similarity
```

不使用：

```
department bonus
query hint
score threshold
LLM generation
```

原因是本项目需要一个清晰的 `embedding_baseline`。如果把 department bonus、query hint 等规则也混入 `embedding_score`，baseline 就不再是纯粹的 embedding baseline，而会变成：

```
embedding similarity + rule-based retrieval heuristic
```

这会让后续和 BGE、BCE、lightweight ML 的比较变得不清楚。

11. Lightweight ML 使用的特征

在 `lightweight_ml` 中，模型不会直接读取原始中文文本，而是使用人工设计的数值特征：

```
embedding_score
keyword_overlap
department_match
question_length
context_length
length_ratio
```

这些特征的含义如下：

特征	含义
<code>embedding_score</code>	原始 RAG embedding cosine similarity
<code>keyword_overlap</code>	question 与 candidate_context 的字符/关键词重合程度
<code>department_match</code>	<code>question_department == department</code> 时为 1，否则为 0
<code>question_length</code>	question 文本长度，经过归一化
<code>context_length</code>	candidate_context 文本长度，经过归一化
<code>length_ratio</code>	question 与 context 长度的相对比例，避免长度极端不平衡

训练时，模型学习的是：

features → label

也就是学习什么样的特征组合更可能对应 `label=1`。

需要注意的是，以下字段不能作为 ML feature：

label
source_type
qid
context_id

其中 `label` 是监督学习答案，不能作为输入；`source_type` 中包含 `positive / hard_negative / random_negative`，如果给模型使用，会造成 data leakage。

12. 后续方法如何使用 candidates.csv

生成 `candidates.csv` 后，四种方法都会读取同一个文件：

`embedding_baseline`：
直接按 `embedding_score` 在每个 `qid` 内部排序。

`bge`：
读取 `question` 和 `candidate_context`，用 BGE reranker 打分并排序。

`bce`：
读取 `question` 和 `candidate_context`，用 BCE reranker 打分并排序。

`lightweight_ml`：
读取 `candidates.csv`，提取数值特征，训练轻量 ML 模型，然后输出 `relevance score` 并排序。

所有方法输出格式必须统一：

qid, context_id, score, rank, method, latency_ms

其中：

score 越高表示越相关
rank 必须在每个 `qid` 内部单独排序
method 只能是 `embedding_baseline / bge / bce / lightweight_ml`

13. 评估方式

最终评估时，将每个方法的排序结果与 `candidates.csv` 中的 `label` 合并。

主要指标包括：

```
Precision@1  
Precision@3  
MRR  
Average latency
```

含义是：

```
Precision@1: 正确 context 是否排在第 1 位  
Precision@3: 正确 context 是否进入前 3  
MRR: 正确 context 排名越靠前，分数越高  
Average latency: 平均打分耗时
```

如果时间允许，也可以加入：

```
Precision@5  
NDCG@5  
Lightweight ML 的 Accuracy / Precision / Recall / F1
```

14. 当前生成命令

在项目根目录运行：

```
python -m src.build_dataset --num-questions 100 --hard-negatives 4 --random-negatives 5 --seed 42
```

预期输出：

```
data/processed/candidates.csv
```

预期规模：

```
100 questions × 10 candidates = 1000 rows
```

其中每个 question 包含：

```
1 positive
4 hard_negative
5 random_negative
```

15. 总结

本项目的 candidate generation 本质上是将原始医疗 QA 数据转化为一个标准 reranking benchmark。

原始数据是：

```
question → answer
```

转换后是：

```
question → multiple candidate contexts → relevance label
```

这样我们就可以独立比较不同 reranker 的 context selection 能力，而不必依赖完整 RAG 生成系统。

该设计的核心原则是：

```
统一 candidates.csv
统一 label 规则
统一输出格式
统一评估指标
```

这样才能保证 embedding baseline、BGE、BCE 和 lightweight ML 的实验结果可以公平比较。